

onto-practice

Прототип системы автоматической проверки приказов о направлении на практику студентов на соответствие регламенту кафедры через OWL-онтологию и встроенный ризонер.

Регламент-источник: «Методические рекомендации по организации и проведению практики обучающихся ФИТ» НГУ (направления 09.03.01 / 09.04.01).

1. Что делает система

Принимает PDF, DOCX или TXT с приказом ректора о направлении студентов на практику и выдаёт HTML-отчёт: какие пункты регламента нарушены, по каким студентам, со ссылкой на пункт регламента.

- Извлечение фактов из документа делает LLM (через сервис OpenRouter).
- Применение правил делает классический OWL DL-ризонер HermiT.
- Сами правила хранятся в OWL-файле — онтологии регламента, которую можно редактировать в Protégé без правок кода ядра.

2. Архитектура — два пайплайна

Система состоит из двух независимых процессов. Они идут по разному поводу, разными командами, имеют разные SLA.

2.1. Build-time — построение онтологии (один раз на регламент)

PDF регламента → текст → LLM извлекает черновик онтологии (классы, свойства, правила) → аналитик дорабатывает черновик в Protégé → итоговый practice.owl, который кладётся в regulations/.

Этап выполняется один раз на каждый регламент. Чтобы добавить регламент другой кафедры — кладём ещё один .owl в ту же папку.

2.2. Runtime — проверка одного приказа

Пользователь загружает PDF/DOCX/TXT приказа → пайплайн извлекает данные (OCR + структурирование текстом) → факты подгружаются в копию онтологии как individuals → HermiT прогоняет правила → отчёт сохраняется и отображается.

3. Стек технологий

Компонент	Технология
Веб-сервер	FastAPI + Jinja2 + минимальный CSS
Онтология	OWL 2 / RDF/XML, читается через owlready2
DL-ризонер	HermiT (Java, поднимается owlready2 автоматически)

Компонент	Технология
Извлечение данных из PDF/DOCX-файлов	PyMuPDF + Microsoft Word OCR (опционально) + python-docx
Удалённый OCR (для сканов)	OpenRouter, модель baidu/qianfan-ocr-fast
Структурирование текста в JSON	OpenRouter, модель inclusionai/ling-2.6-1t (с fallback-цепочкой)
Управление LLM-ключами	in-memory pool с ротацией на дневной лимит и upstream rate-limit

4. Правила проверки (R-01..R-07)

Правила хранятся в онтологии как индивиды класса ViolationRule. Проверки реализованы в src/core/checker.py.

ID	Правило	Источник в регламенте
R-01	Заполнены все обязательные поля по студенту	Приложения 1-3
R-02	Программа подготовки $\in \{09.03.01, 09.04.01\}$	стр. 2
R-03	Дата начала $\leq$ даты окончания практики	здравый смысл
R-04	От даты приказа до начала практики ≥ 30 дней	стр. 9
R-05	Назначен один руководитель от НГУ	стр. 6
R-06	Номера зачётных книжек уникальны	здравый смысл
R-07	Указан тип практики (учебная / производственная)	стр. 3

5. Инструкция: проверка приказа

Аудитория: сотрудник учебно-методического отдела.

Шаг 1. Открыть веб-приложение

Перейти по адресу системы. На главной странице — форма загрузки.

Шаг 2. Подготовить файл приказа

Поддерживаются три формата:

- .docx или .txt — самое лучшее качество. Если у вас сканированный PDF — откройте его в Word, Word автоматически распознает текст, сохраните как .docx и используйте его.
- .pdf — система прогонит свой OCR; качество распознавания русских фамилий на сканах низкого разрешения хуже, чем у Word.

Шаг 3. Загрузить и подождать

- DOCX/TXT — обработка ~30-60 секунд.
- PDF (скан) — 2-7 минут на бесплатных моделях.
- Страница автоматически обновляется. По готовности появится отчёт.

Шаг 4. Прочитать отчёт

Отчёт состоит из четырёх блоков:

- Сводка — число студентов, число нарушений, статус ризонера.
- Параметры приказа — что распарсено: номер, период, направление, тип.
- Нарушения — таблица: код правила, студент, описание, ссылка на пункт регламента. Ошибки выделены красной полосой слева, предупреждения — оранжевой.
- Применённые правила — справка по всем R-01..R-07 с описанием и источником в регламенте.

Шаг 5. Что делать с нарушениями

- R-04 (срок): согласовать перенос приказа на более раннюю дату или перенести начало практики.
- R-01 / R-05 (пустые поля): сверить с реальным документом — иногда это OCR-пропуск; если действительно не заполнено — дополнить приказ.
- R-02 (программа): проверить код в шапке приказа — возможно опечатка.

6. Инструкция: обновление онтологии

Аудитория: аналитик / эксперт по регламенту.

Когда нужно обновлять онтологию

- Регламент кафедры обновился — новые правила, новые требования к полям.
- Появилась новая образовательная программа — расширили список допустимых направлений в R-02.
- Поменялась форма приказа — например, добавилась колонка.

Как обновить

- Открыть страницу Онтология → нажать «скачать» рядом с draft.
- Открыть скачанный practice.draft.owl в Protégé.
- Доработать по чек-листу из инструкции /ontology/workflow (она доступна как ссылка на странице Онтология).
- Сохранить как practice.owl в формате RDF/XML.
- Вернуться на страницу Онтология, в форме «Загрузить новую финальную онтологию» выбрать файл и нажать «Загрузить».
- Старая версия автоматически бэкапится в var/backups/.
- Следующая проверка приказа использует уже новую онтологию.

Чтобы получить совершенно новый черновик из обновлённого регламента

Это разовая команда у разработчика — пайплайн читает PDF регламента, вызывает LLM, кладёт черновик в var/drafts/ откуда его можно скачать через интерфейс:

```
python scripts/build_ontology.py путь/к/новому/регламенту.pdf
```

7. Ограничения текущей реализации

Прототип построен на бесплатных моделях OpenRouter — это влияет на качество и скорость:

- Точность распознавания фамилий на скан-PDF — около 50–70%. Word OCR (для DOCX) даёт ~90%. Платные модели типа GPT-5-nano или Mistral OCR дают 95–99%.
- Дневной лимит бесплатного OpenRouter — 50 запросов на ключ. В системе ротация по 5 ключам = 250 запросов в день, ~25 приказов.
- Иногда зависают запросы к перегруженным провайдерам моделей. Логируется, выставлены timeout/retry, в худшем случае — загрузить файл заново.

Что меняется при переходе на платный тариф

Один тот же ключ OpenRouter работает и с бесплатными, и с платными моделями. Достаточно положить кредиты на счёт и поменять две строки в конфиге .env (имена моделей):

```
LLM_MODEL_VISION=mistral/mistral-ocr-latest  
LLM_MODEL_TEXT=openai/gpt-4o-mini
```

Один приказ обходится в 1–2 цента. \$5 хватает на 250–500 приказов. Время обработки — 20–40 секунд. Никаких rate-limit.

Чего нет в прототипе (по техническому заданию)

- Аутентификация и роли — все эндпойнты публичные.
- База данных отчётов — отчёты лежат как JSON-файлы в var/reports/.
- Редактирование онтологии через UI — только через Protégé. Загрузка готового файла через UI — есть.
- Production-deploy — система запускается локально через uvicorn, для теста заказчика выставлена через Cloudflare Quick Tunnel.

8. Локальное развёртывание

Аудитория: разработчик / системный администратор.

8.1. Что нужно из коробки

- Linux / macOS / Windows с Python 3.11 или новее.
- ~3 ГБ свободного места (для venv с зависимостями и онтологии).
- Доступ в интернет к <https://openrouter.ai> (для LLM-вызовов).
- Java 17+ — нужна ризонеру HerMiT. Если в системе Java нет, приложение само установит её при первом запуске через pip-пакет install-jdk в каталог ~/.jdk/.

8.2. Скачать исходники

Архив со всеми файлами проекта доступен по адресу `/docs/onto-practice-source.zip` на работающем сервисе. Распаковать в любую папку, дальше всё запускается из неё.

8.3. Поставить зависимости и запустить

```
cd onto-practice
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -e ".[dev]"

# (если нет java в PATH)
python -c "import jdk; jdk.install('17')"

# .env: положить ключ OpenRouter
cp .env.example .env
# отредактировать .env: подставить OPENROUTER_API_KEYS=sk-or-v1-...

uvicorn src.web.main:app --host 0.0.0.0 --port 8000 --reload
```

Открыть в браузере `http://localhost:8000` — должна появиться форма загрузки приказа.

8.4. Регенерация черновика онтологии (по необходимости)

Если регламент кафедры обновился — регенерировать LLM-черновик:

```
python scripts/build_ontology.py путь/к/regulation.pdf
```

Файл попадёт в `var/drafts/practice.draft.owl` и появится в интерфейсе на странице Онтология. Дальше — доводка в Protégé по разделу 6.

8.5. Проверки и линтер

```
ruff check . # стиль кода
pytest # тесты (опционально, минимально)
```

8.6. Где что лежит

- `regulations/practice.owl` — рабочая онтология регламента (по ней проверяются приказы).
- `.env` — ключи OpenRouter и имена моделей.
- `var/uploads/` — загруженные документы.
- `var/reports/` — отчёты по проверкам в JSON.
- `var/cache/facts/` — кэш извлечённых фактов по SHA256 файла (повторная загрузка идёт мгновенно).
- `var/backups/` — бэкапы предыдущих версий `practice.owl` при загрузке через UI.

8.7. Типичные проблемы

- «HermiT reasoner unavailable: no java»: Java не нашлась. Запустить `python -c "import jdk; jdk.install('17')"` и перезапустить `uvicorn`.

- «All structure models failed»: упёрлись в дневной лимит OpenRouter всеми ключами. Подождать сутки или пополнить счёт (\$5 даёт 1000 free req/day или платные модели за центы).
- «No endpoints found for <model>»: модель сняли с free tier. Поменять LLM_MODEL_TEXT или LLM_MODEL_VISION в .env на актуальную бесплатную модель из https://openrouter.ai/models?max_price=0.

9. Структура проекта

```
src/builder – build-time pipeline (PDF регламента → черновик онтологии)
src/extractors – runtime pipeline (документ приказа → факты)
src/core – загрузка онтологии, инъекция фактов, ризонер, отчёт
src/web – FastAPI + Jinja2 шаблоны
regulations/ – итоговые .owl-файлы регламентов (плагины ядра)
var/drafts – LLM-черновики до доводки в Protégé
var/uploads, var/reports, var/cache – runtime-данные
docs – requirements, architecture, эта документация
```

10. Контрольный пример

Демо-приказ — № 0145-2 от 22.01.2026 о направлении студентов 3 курса ПИиКН на учебную практику с 02.02.2026 по 28.05.2026.

Между датой приказа (22.01) и началом практики (02.02) всего 11 дней, регламент требует не менее 30. Система должна срабатывать R-04 на этом приказе — это поддерживает любая загрузка демо-документа.